

Bootstrap — Topics 23-31: Alerts, Badges, Breadcrumbs, Pagination, Progress Bars, Spinners, Collapse, Accordion, Carousel

Study paper format: notes (Roman Urdu + English technical terms), har component ke peeche ka CSS/JS mechanism, + active recall Q&A at the end.

23. Alerts

Alert ek **contextual message box** hai — success, error, warning, ya info dikhane ke liye.

```
html

<div class="alert alert-success" role="alert">
  Operation successful!
</div>

<div class="alert alert-danger" role="alert">
  Something went wrong.
</div>
```

Colors: `alert-primary`, `alert-secondary`, `alert-success`, `alert-danger`, `alert-warning`, `alert-info`, `alert-light`, `alert-dark`.

Peeche ki CSS: `.alert` class padding, border-radius, aur border deti hai. `.alert-success` waghera sirf `background-color`, `border-color`, aur `color` set karte hain (halki-si tinted background, dark text same color family mein) — ye sab CSS variables/utility classes ka combination hai, koi JS involved nahi basic alert mein.

Dismissible Alerts

```
html
```

```
<div class="alert alert-warning alert-dismissible fade show" role="alert">
  This is a dismissible alert.
  <button type="button" class="btn-close" data-bs-dismiss="alert" aria-label="Close"></button>
</div>
```

- `.alert-dismissible` → extra padding right side par deta hai (close button ke liye space)
- `.btn-close` → chota "X" icon button (background-image se banta hai, koi text nahi)
- `data-bs-dismiss="alert"` → Bootstrap ke JS ko batata hai ke is button ko click karne par parent `.alert` ko remove/hide karna hai
- `.fade` + `.show` → CSS transition classes; `.fade` opacity transition define karta hai, `.show` opacity ko 1 kar deta hai. Jab alert dismiss hota hai, Bootstrap JS `.show` class hata deta hai jisse CSS transition se alert **fade out** hota hai, phir element DOM se remove ho jata hai.

Mechanism summary: Close button click → Bootstrap JS listener fire hota hai → `.show` class remove → CSS transition (opacity fade) chalti hai → element hide/remove ho jata hai.

24. Badges

Badge ek chota **label/counter** hota hai jo kisi element ke sath extra info dikhata hai (count, status).

```
html

<span class="btn btn-primary">
  Messages <span class="badge bg-secondary">4</span>
</span>
```

Peeche ki CSS: `.badge` `display: inline-block`, chota `font-size` (parent se relatively chota), `padding`, aur `border-radius` deta hai — isliye badge apne parent text ke sath inline fit ho jata hai, lekin visually alag box ki tarah stand out karta hai. Color `bg-*` utility classes se aata hai (jaise `bg-primary`, `bg-danger`).

Notification Badges (Positioned)

html

```
<button type="button" class="btn btn-primary position-relative">
  Inbox
  <span class="position-absolute top-0 start-100 translate-middle badge rounded-pill">
    99+
  </span>
</button>
```

Peeche ki CSS/mechanism:

- Parent button par `position-relative` — isse ye ek **positioning context** ban jata hai
- Badge par `position-absolute` — isse badge parent ke relative position hota hai (poori page ke relative nahi)
- `top-0 start-100` — badge ko parent ke **top-right corner** par le jata hai (top edge par, aur horizontally 100% yani right edge par)
- `translate-middle` — CSS `transform: translate(-50%, -50%)` apply karta hai, jisse badge apne hi center point par exactly wahi corner par align ho jata hai (bina isse, badge corner se thora bahar/andar hota)
- `rounded-pill` → bara `border-radius` jisse number ke around perfect pill/circle shape ban jaye

25. Breadcrumbs

Breadcrumb user ko batata hai ke wo site ki hierarchy mein **kahan hai** (navigation trail).

```
html

<nav aria-label="breadcrumb">
  <ol class="breadcrumb">
    <li class="breadcrumb-item"><a href="#">Home</a></li>
    <li class="breadcrumb-item"><a href="#">Library</a></li>
    <li class="breadcrumb-item active" aria-current="page">Data</li>
  </ol>
</nav>
```

Peeche ki CSS: `.breadcrumb` `<ol>` ko `display: flex` deta hai (isliye items horizontally line mein aate hain) aur default list bullets/numbering remove karta hai. Har `.breadcrumb-item` CSS `::before` **pseudo-element** use karta hai jo automatically ek separator (default: `/`) insert karta hai — is separator ko HTML mein manually likhne ki zaroorat nahi, ye pure CSS se generate hota hai. Last item par `.active` class use hoti hai jo current/non-clickable page ko represent karti hai (usually muted color, no link).

26. Pagination

Pagination multi-page content (jaise search results) ke beech navigate karne ke liye numbered links deta hai.

```
html
```

```
<nav aria-label="Page navigation">
  <ul class="pagination">
    <li class="page-item"><a class="page-link" href="#">Previous</a></li>
    <li class="page-item"><a class="page-link" href="#">1</a></li>
    <li class="page-item active"><a class="page-link" href="#">2</a></li>
    <li class="page-item"><a class="page-link" href="#">3</a></li>
    <li class="page-item"><a class="page-link" href="#">Next</a></li>
  </ul>
</nav>
```

Peeche ki CSS: `.pagination` bhi `display: flex` use karta hai `<ul>` par (items horizontal). Har `.page-link` ko border, padding, aur negative `margin-left` diya jata hai — negative margin se adjacent links ke borders **overlap** ho jate hain, jisse ek continuous connected border ka look banta hai (bina double-thick borders ke beech mein). `.active` class current page ko highlighted background deti hai. `.disabled` class (jaise "Previous" jab first page par ho) link ko grayed-out aur non-clickable bana deti hai (`pointer-events: none` + muted color).

Sizes: `.pagination-lg`, `.pagination-sm` — poore `<ul>` par lagti hain, jaise button groups.

27. Progress Bars

Progress bar kisi task ke **completion percentage** ko visually show karta hai.

```
html

<div class="progress" role="progressbar" aria-valuenow="60" aria-valuemin="0"
  <div class="progress-bar" style="width: 60%">60%</div>
</div>
```

Peeche ki CSS/mechanism (ye important hai):

- `.progress` outer container hai — fixed height, light gray background, `overflow: hidden` (taake andar wali bar rounded corners se bahar na nikle)
- `.progress-bar` andar wala colored `<div>` hai jiski `width` inline style se set hoti hai — yehi asal mechanism hai! `width: 60%` matlab bar apni parent width ka 60% cover karega. Baqi 40% gray background (parent ka) nazar aata hai.
- Percentage change karna ho to bas `width` value JS se dynamically update kar do — Bootstrap khud koi calculation nahi karta, developer ko value set karni parti hai
- Color ke liye `bg-success`, `bg-danger`, etc. `.progress-bar` par lagai ja sakti hain

Striped

```
html
<div class="progress">
  <div class="progress-bar progress-bar-striped" style="width: 50%"></div>
</div>
```

Peeche ki CSS: `progress-bar-striped` ek CSS `linear-gradient` background use karta hai jo diagonal alternating stripes banata hai (transparent aur semi-transparent white ke beech), `background-size` fix kar ke pattern repeat karta hai.

Animated

```
html
<div class="progress">
  <div class="progress-bar progress-bar-striped progress-bar-animated" style="width: 50%"></div>
```

Peeche ki CSS: `progress-bar-animated` ek CSS `@keyframes` animation add karta hai jo `background-position` ko continuously shift karta hai — isse stripes **moving/scrolling** effect dete hain (left se right infinite loop). Ye sirf striped ke sath meaningful hai (animated akele bina stripes ke visually kuch nahi karega).

28. Spinners

Spinner loading state indicate karne ke liye hota hai (jab data fetch ho raha ho).

```
html

<div class="spinner-border text-primary" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
```

Peeche ki CSS: `.spinner-border` ek circular element hai jiski border ka **ek side transparent** hota hai (baqi sides colored) — CSS `@keyframes` animation isse continuously `rotate(360deg)` karta hai, jisse ek "spinning circle" ka illusion banta hai. `role="status"` aur andar ka `.visually-hidden` text (screen se hidden lekin screen-readers ko sunai deta hai) accessibility ke liye hote hain.

Growing Spinner (Loading Indicator variant)

```
html

<div class="spinner-grow text-success" role="status">
  <span class="visually-hidden">Loading...</span>
</div>
```

Peeche ki CSS: `.spinner-grow` border-based rotation ki bajaye element ki `scale()` **transform** ko `@keyframes` ke zariye 0 se 1 tak animate karta hai, aur sath opacity bhi fade karta hai — isse ek "pulsing/growing dot" effect banta hai, spinning wheel nahi.

Size: `.spinner-border-sm`, `.spinner-grow-sm` chota version.

29. Collapse

Collapse content ko **show/hide** karne ka mechanism hai — smooth height transition ke sath.

```
html

<button class="btn btn-primary" data-bs-toggle="collapse" data-bs-target="#collapseExample" type="button">
  Toggle Content
</button>

<div class="collapse" id="collapseExample">
  <div class="card card-body">
    Ye content collapse/expand hoga.
  </div>
</div>
```

Peeche ki CSS/mechanism:

- Default `.collapse` class element ko `display: none` kar deti hai (hidden state)
- Button click hone par, Bootstrap JS (`data-bs-toggle="collapse"` + `data-bs-target`) target element par `.collapsing` class temporarily add karta hai, jo `height` ko `0` se actual content height tak **CSS transition** karta hai (smooth animation)
- Transition complete hone ke baad, JS `.collapsing` ko hata kar `.show` class add kar deta hai, jo `display: block` set karti hai (final open state)
- Dobara click karne par reverse process hota hai — `.show` hat jati hai, height transition reverse mein chalti hai, phir `display: none` ho jata hai

Key point: Ye pura height-animation JS ke bina possible nahi hai kyunke CSS akele `auto` height ko directly animate nahi kar sakti — Bootstrap JS actual pixel height calculate kar ke transition karvata hai.

30. Accordion (FAQ Sections)

Accordion multiple collapse sections ka **group** hai jahan aksar sirf ek section ek time par open rehta hai.

html

```
<div class="accordion" id="faqAccordion">

  <div class="accordion-item">
    <h2 class="accordion-header">
      <button class="accordion-button" type="button" data-bs-toggle="collapse">
        What is Bootstrap?
      </button>
    </h2>
    <div id="faq1" class="accordion-collapse collapse show" data-bs-parent="#faqAccordion">
      <div class="accordion-body">
        Bootstrap is a CSS framework...
      </div>
    </div>
  </div>

  <div class="accordion-item">
    <h2 class="accordion-header">
      <button class="accordion-button collapsed" type="button" data-bs-toggle="collapse">
        Why use Bootstrap?
      </button>
    </h2>
    <div id="faq2" class="accordion-collapse collapse" data-bs-parent="#faqAccordion">
      <div class="accordion-body">
        It speeds up development...
      </div>
    </div>
  </div>

</div>
```

Peeche ki CSS/mechanism:

- Accordion asal mein **Collapse component ka hi organized wrapper** hai — har item ke andar wahi `.collapse` mechanism chal raha hai jo Topic 29 mein explain hua
 - `data-bs-parent="#faqAccordion"` sabse important attribute hai — ye Bootstrap JS ko batata hai ke ye sab collapse items **ek hi group** ka hissa hain. Jab ek item open hota hai, JS automatically doosre sab open items ko close kar deta hai (accordion ka signature behavior)
 - `.accordion-button` par `::after` pseudo-element se ek **arrow icon** (background-image) hota hai — CSS `transform: rotate(180deg)` se ye arrow flip hota hai jab button expanded state mein ho (non-`.collapsed` class), aur jab collapsed ho to normal position mein rehta hai
 - `.collapsed` class button par tab lagti hai jab uska corresponding section band ho — ye styling ke liye (arrow direction, background color) use hoti hai
-

31. Carousel (Image Sliders)

Carousel images/content ka ek **rotating slideshow** hai.

html

```

<div id="myCarousel" class="carousel slide" data-bs-ride="carousel">

  <div class="carousel-indicators">
    <button data-bs-target="#myCarousel" data-bs-slide-to="0" class="active">
    <button data-bs-target="#myCarousel" data-bs-slide-to="1"></button>
  </div>

  <div class="carousel-inner">
    <div class="carousel-item active">
      
    </div>
    <div class="carousel-item">
      
    </div>
  </div>

  <button class="carousel-control-prev" type="button" data-bs-target="#myCa
    <span class="carousel-control-prev-icon"></span>
  </button>
  <button class="carousel-control-next" type="button" data-bs-target="#myCa
    <span class="carousel-control-next-icon"></span>
  </button>

</div>

```

Peeche ki CSS/mechanism (detail se):

- `.carousel-inner` ek container hai jispar `overflow: hidden` hota hai — isliye sirf ek slide ek time par visible hoti hai, baqi clip ho jati hain
- Har `.carousel-item` by default `display: none` hota hai; sirf `.active` wala item `display: block` hota hai

- **Slide transition:** jab `.carousel-slide` (via `.slide` class on parent) mode active ho, to slide change hone par Bootstrap JS naye item ko `position: absolute` kar ke current item ke sath sath layer karta hai, aur CSS `transform: translateX()` transition use karta hai — purana slide left/right mein slide-out hota hai, naya slide simultaneously slide-in hota hai. Ye sab `transition` property se smooth hota hai, harsh jump nahi
- `data-bs-ride="carousel"` → automatically slides ko har kuch second baad JS timer (`setInterval`)-jaisi logic) se change karta hai
- `.carousel-control-prev` / `.carousel-control-next` → click hone par JS `.active` class ko current se next/previous `.carousel-item` par move karta hai, jisse transition trigger hoti hai
- `.carousel-indicators` → chote dots/bars jo current slide show karte hain; `.active` class current dot ko highlight karti hai; click karne par `data-bs-slide-to="N"` directly Nth slide par jump karva deta hai

Summary: Carousel = `.carousel-inner` (clipping window) + har slide ka `display: none/block` toggle + `.active` slide par `position: absolute` aur `transform: translateX()` transition, sab JS se orchestrate hota hai taake smooth sliding effect bane.

Cheat Sheet

Component	Key Classes	Underlying Mechanism
Alert	<code>alert</code> , <code>alert-{color}</code> , <code>alert-dismissible</code> , <code>btn-close</code> , <code>fade show</code>	<code>.fade</code> / <code>.show</code> = opacity CSS transition; JS removes element after dismiss
Badge	<code>badge</code> , <code>bg-{color}</code> , <code>rounded-pill</code>	<code>display:inline-block</code> + small font-size
Notification Badge	<code>position-relative</code> / <code>position-absolute</code> , <code>top-0 start-100</code> , <code>translate-middle</code>	<code>transform: translate(-50%, -50%)</code> centers badge exactly on corner
Breadcrumb	<code>breadcrumb</code> , <code>breadcrumb-item</code> , <code>active</code>	Flex list; separator via CSS <code>::before</code> pseudo-element
Pagination	<code>pagination</code> , <code>page-item</code> , <code>page-link</code> , <code>active</code> , <code>disabled</code>	Flex list; negative margin overlaps borders for connected look
Progress	<code>progress</code> , <code>progress-bar</code> , <code>progress-bar-striped</code> , <code>progress-bar-animated</code>	Inline <code>width</code> = fill %; stripes = <code>linear-gradient</code> ; animation = <code>@keyframes</code> shifting background-position
Spinner	<code>spinner-border</code> , <code>spinner-grow</code>	<code>@keyframes</code> rotate (border) or scale+opacity (grow)
Collapse	<code>collapse</code> , <code>collapsing</code> , <code>show</code> , <code>data-bs-toggle="collapse"</code>	JS animates height 0→auto via <code>.collapsing</code> , then swaps to <code>display:block</code> via <code>.show</code>

Component	Key Classes	Underlying Mechanism
Accordion	<code>accordion</code> , <code>accordion-item</code> , <code>accordion-button</code> , <code>accordion-collapse</code> , <code>data-bs-parent</code>	Wraps Collapse; <code>data-bs-parent</code> makes JS auto-close sibling items
Carousel	<code>carousel</code> , <code>carousel-inner</code> , <code>carousel-item</code> , <code>active</code> , <code>carousel-indicators</code> , <code>carousel-control-prev/next</code>	<code>overflow: hidden</code> window; <code>display none/block</code> + <code>transform: translateX()</code> transition between active items

Practice Exercises

1. Ek dismissible success alert banao jo user action ke baad show ho aur close button se dismiss ho sake.
2. Ek notification icon banao jisme `position-absolute` badge top-right corner par ho (jaise cart items count).
3. Ek breadcrumb banao: Home → Courses → Bootstrap → Grid System (last item `active`).
4. Ek pagination banao 5 pages ke sath, current page ko `active` aur "Previous" ko `disabled` (jab page 1 ho) mark karo.
5. Ek animated striped progress bar banao jo 80% show kare.
6. Ek loading spinner banao (`spinner-border`) jo button ke andar ho — jaise "Submitting..." state.
7. Ek FAQ accordion banao 3 questions ke sath, sirf ek time ek answer open rahe.
8. Ek 3-slide image carousel banao with indicators aur prev/next controls.

Active Recall — Alerts, Badges, Breadcrumbs, Pagination, Progress, Spinners, Collapse, Accordion, Carousel

| Khud se answer karne ki koshish karo, phir neech scroll karke check karo.

Q1. Dismissible alert mein `.fade` aur `.show` classes ka kya kaam hai?

Q2. Notification badge ko parent ke exact corner par center karne ke liye `translate-middle` kya CSS transform apply karta hai, aur kyun zaroori hai?

Q3. Breadcrumb separators (`/`) HTML mein manually likhe jate hain ya CSS se generate hote hain?

Q4. Pagination links ka connected border look kaise achieve hota hai?

Q5. Progress bar ki fill percentage kis property se control hoti hai, aur Bootstrap khud calculate karta hai ya developer set karta hai?

Q6. `progress-bar-striped` aur `progress-bar-animated` mein kya farq hai — animated akele (bina striped ke) kya effect dega?

Q7. `spinner-border` aur `spinner-grow` ka animation mechanism kaise different hai?

Q8. Collapse component mein height animation kaise achieve hoti hai — kya CSS akele `auto` height ko animate kar sakti hai?

Q9. Accordion mein `data-bs-parent` attribute ka kya kaam hai?

Q10. Carousel mein ek time par sirf ek slide kaise visible rehti hai (konsi CSS properties involved hain)?

Q11. Carousel slide transition ke waqt konsi CSS property use hoti hai smooth sliding effect ke liye?

Answers

<details> <summary>Answers dekhne ke liye click karein</summary>

A1. `.fade` element par ek opacity-based CSS transition define karta hai. `.show` opacity ko `1` set karta hai (visible state). Jab alert dismiss hota hai, JS `.show` class hata deta hai jisse transition ke through element **fade out** hota hai, phir DOM se remove ho jata hai.

A2. `translate-middle` `transform: translate(-50%, -50%)` apply karta hai — ye element ko uski apni width/height ke 50% se upar-left shift karta hai, jisse element ka **center point** exactly us position par align ho jata hai jahan `top-0 start-100` ne use rakha tha. Iske bina, badge corner se thora bahar ya andar dikhta.

A3. CSS se generate hote hain — har `.breadcrumb-item` `::before` pseudo-element use karta hai jo automatically separator insert karta hai; ye HTML mein manually nahi likha jata.

A4. Har `.page-link` par negative `margin-left` lagti hai jo adjacent links ke borders ko overlap kar deti hai, jisse ek continuous single border ka look banta hai (double-thick borders nahi dikhte).

A5. `.progress-bar` ki inline `width` (percentage mein) fill control karti hai. Bootstrap khud koi calculation nahi karta — developer ko manually ya JS se width value set/update karni parti hai.

A6. `progress-bar-striped` diagonal stripes ka static pattern deta hai (CSS `linear-gradient` se). `progress-bar-animated` un stripes ke `background-position` ko `@keyframes` se continuously shift karta hai (moving effect). Animated class akele (bina striped ke) visually koi noticeable effect nahi degi, kyunke animate karne ke liye stripes pattern hona zaroori hai.

A7. `spinner-border` ek circular border ko continuously `rotate(360deg)` karta hai (ek side transparent hone ki wajah se spinning wheel effect). `spinner-grow` element ko `scale()` transform se 0 se 1 tak grow karta hai sath opacity fade ke sath — pulsing dot jaisa effect, rotation nahi.

A8. JS actual target element ki content height calculate karta hai aur `.collapsing` class ke through height ko `0` se us calculated height tak transition karta hai; transition

complete hone par `.show` class (`display: block`) laga di jati hai. CSS akele `auto` height ko directly smoothly animate nahi kar sakti — isliye JS involvement zaroori hai.

A9. `data-bs-parent` Bootstrap JS ko batata hai ke sab sibling collapse items **ek hi accordion group** ka hissa hain. Jab ek item open hota hai, JS automatically baqi open items ko close kar deta hai — yehi accordion ka "sirf ek time ek open" behavior hai.

A10. Har `.carousel-item` default `display: none` hota hai; sirf `.active` class wala item `display: block` hota hai. Sath hi `.carousel-inner` par `overflow: hidden` hota hai jo extra/off-screen content ko clip kar deta hai.

A11. `transform: translateX()` CSS transition use hoti hai — purana slide ek direction mein slide-out hota hai aur naya slide dusri direction se slide-in hota hai, dono simultaneously animate hote hain smooth horizontal movement ke liye.

</details>